

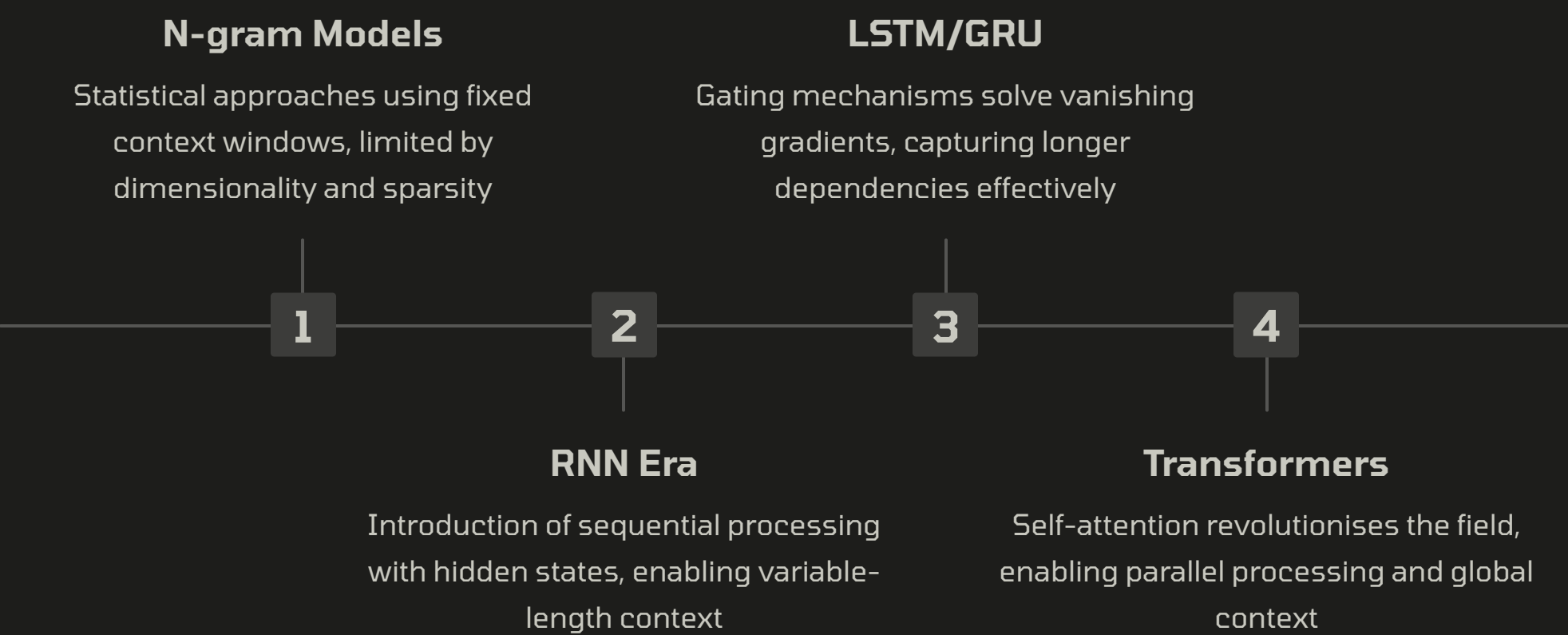
Advanced Concepts in Language Modeling, Image Captioning, and Probabilistic Graphical Models

This document explores cutting-edge techniques and fundamental principles that power modern artificial intelligence systems. From the probabilistic foundations of language modeling to the sophisticated attention mechanisms that enable machines to understand both text and images, we journey through the mathematical and computational frameworks that define contemporary machine learning. We examine how neural architectures bridge vision and language, how attention mechanisms revolutionised sequence-to-sequence learning, and how probabilistic graphical models provide elegant solutions to complex inference problems. Each section combines theoretical rigour with practical implementation guidance, offering both conceptual understanding and actionable code examples.

Language Modeling: Foundations and Modern Architectures

Language modeling represents one of the most fundamental tasks in natural language processing, concerned with predicting the probability distribution of word sequences. At its core, a language model assigns probabilities to sequences of words, enabling applications ranging from predictive text and speech recognition to sophisticated text generation and machine translation systems. The evolution of language modeling techniques mirrors the broader development of machine learning itself, from simple statistical approaches to complex neural architectures.

Early language models relied on n-gram statistical methods, which estimated the probability of a word based on the preceding n-1 words. Whilst computationally efficient, these models suffered from the curse of dimensionality and failed to capture long-range dependencies in text. The neural revolution transformed this landscape dramatically. Recurrent Neural Networks (RNNs) introduced the ability to maintain hidden states across sequences, though they struggled with vanishing gradients over long sequences. Long Short-Term Memory (LSTM) networks addressed this limitation through gating mechanisms that selectively retain or forget information.



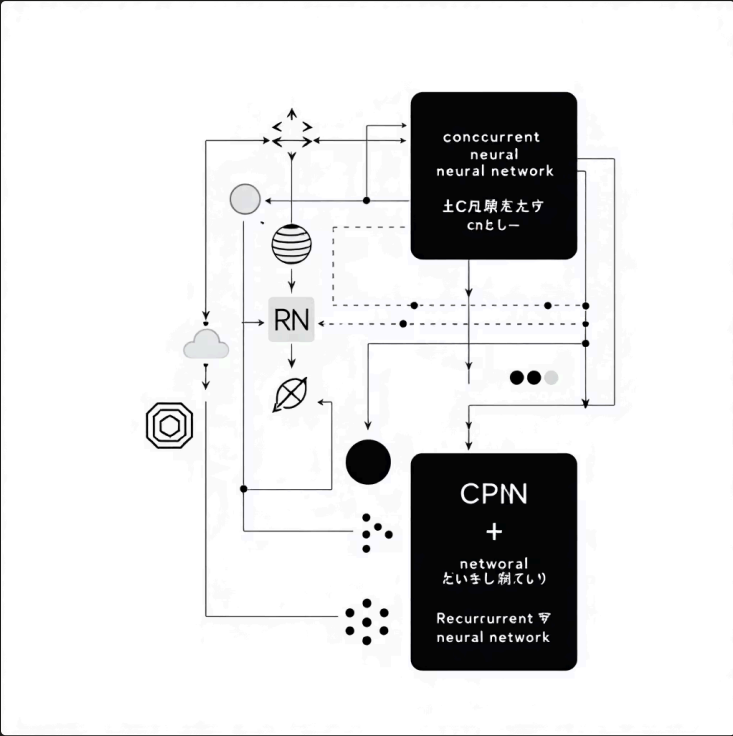
The Transformer architecture, introduced in 2017, fundamentally revolutionised language modeling by eschewing recurrence entirely in favour of self-attention mechanisms. This innovation allows the model to capture long-range dependencies efficiently whilst enabling parallel processing during training. Transformers compute attention scores between all pairs of positions in a sequence, allowing each word to directly attend to any other word regardless of distance. This architectural choice has proven extraordinarily powerful, forming the foundation for breakthrough models like BERT, GPT, and their successors.

Modern Transformers employ multi-head attention, where multiple attention mechanisms operate in parallel, each learning to focus on different aspects of the input. Combined with position encodings, feedforward networks, and layer normalisation, these models achieve state-of-the-art performance across virtually all language understanding and generation tasks. The attention layers enable context-aware word prediction by computing weighted combinations of value vectors, where weights are determined by the relevance of each position to the current prediction task. This mechanism provides both powerful representational capacity and a degree of interpretability, as attention weights can be visualised to understand which parts of the input influence particular predictions.

Image Captioning: Bridging Vision and Language with Attention

Image captioning represents a fascinating confluence of computer vision and natural language processing, tasking models with generating coherent, semantically accurate natural language descriptions of visual content. This multimodal challenge requires systems to not only recognise objects, scenes, and relationships within images but also to articulate these observations in fluent, contextually appropriate language. The problem demands sophisticated integration of visual feature extraction and sequential language generation capabilities.

The typical image captioning pipeline begins with a convolutional neural network (CNN) that extracts rich visual features from the input image. Architectures like ResNet, Inception, or EfficientNet process the image through successive convolutional layers, capturing hierarchical representations from low-level edges and textures to high-level semantic concepts. These visual features are then passed to a sequence generation model—traditionally an RNN or LSTM, though increasingly a Transformer—that produces the caption word by word.



Visual Attention Mechanism

Attention dynamically highlights relevant image regions at each decoding step, mimicking human visual attention patterns

Multi-Head Architecture

Multiple attention heads capture diverse visual aspects simultaneously, enriching contextual understanding

Transformer-Based Models

Modern architectures leverage self-attention for both visual encoding and language generation

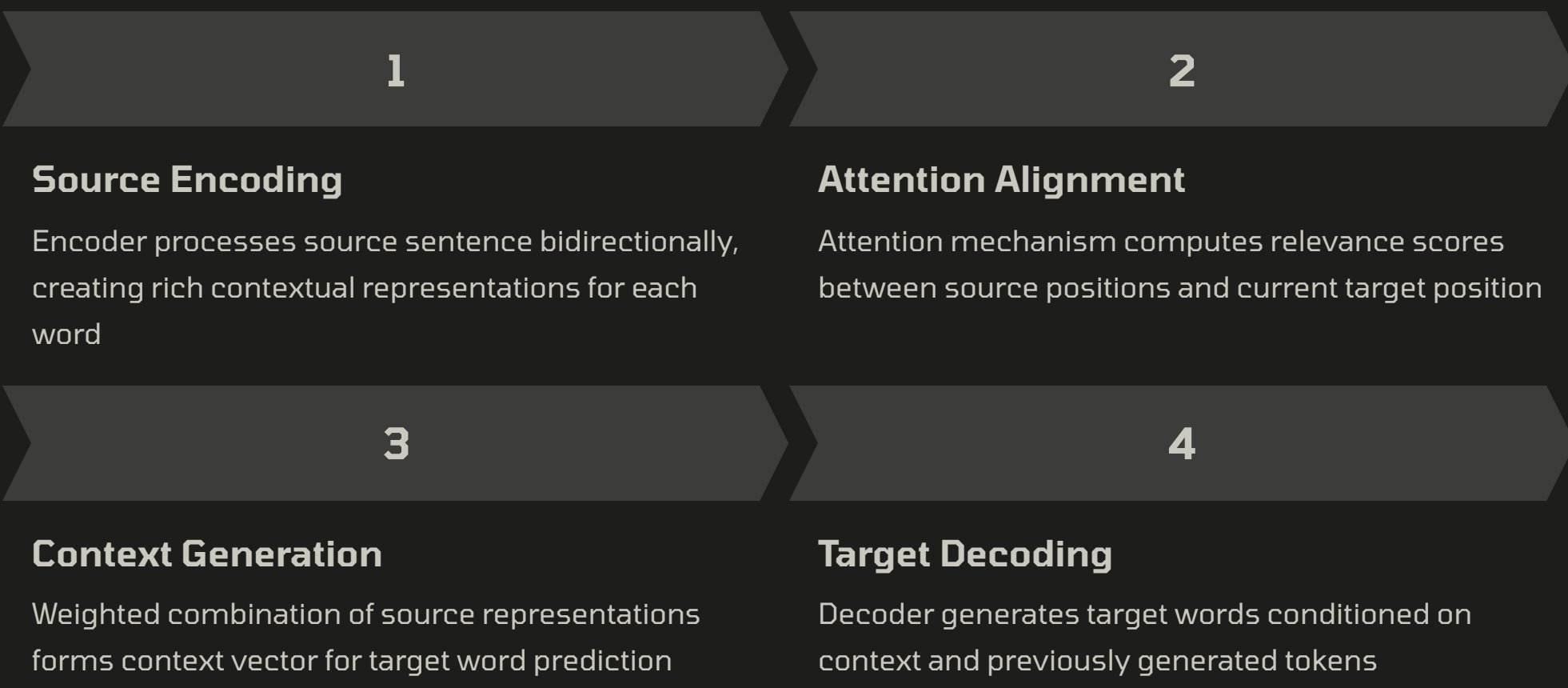
Attention mechanisms have proven transformative for image captioning, addressing a fundamental limitation of early encoder-decoder architectures: the compression of all visual information into a single fixed-length vector. Instead, attention allows the decoder to dynamically focus on relevant regions of the image at each step of caption generation. When generating the word "cat", for instance, the model attends strongly to image regions containing the feline subject, whilst words like "sitting" direct attention to posture-relevant areas. This selective focus dramatically improves both caption accuracy and semantic coherence.

Recent advances have seen Transformer-based captioning models outperform traditional RNN approaches by leveraging multi-head attention for richer context understanding. These models treat image regions as a sequence of visual "tokens", applying self-attention to capture relationships between different parts of the image before cross-attention links visual and linguistic modalities. Challenges remain, however, including maintaining semantic consistency across longer captions, developing robust multilingual captioning systems, and reasoning about complex scenes involving multiple objects, actions, and subtle relationships. Addressing these challenges requires innovations in both architectural design and training methodologies, including better handling of visual reasoning and incorporation of external knowledge sources.

Machine Translation and the Role of Attention Mechanisms

Machine translation—the automatic conversion of text from one language to another—stands as one of the most commercially significant and technically challenging applications of natural language processing. The field has undergone a dramatic transformation over the past decade, evolving from rule-based and phrase-based statistical systems to sophisticated neural architectures that achieve near-human translation quality for many language pairs. This evolution reflects broader trends in machine learning towards end-to-end differentiable systems that learn representations directly from data.

Neural Machine Translation (NMT) employs encoder-decoder architectures where the encoder processes the source sentence into a continuous representation, and the decoder generates the target translation. Early sequence-to-sequence models compressed the entire source sentence into a single fixed-length vector, creating an information bottleneck that limited translation quality, particularly for longer sentences. This architectural constraint meant that all information about the source sentence—regardless of length or complexity—had to be captured in a single vector representation, inevitably leading to information loss.



The introduction of attention mechanisms by Bahdanau et al. revolutionised neural machine translation by allowing the decoder to selectively focus on relevant parts of the source sentence at each generation step. The attention mechanism computes alignment scores between each source position and the current decoder state, then uses these scores to create a weighted combination of source representations. Mathematically, if $h_1, \dots, h_n$ are encoder hidden states and s_t is the decoder state at time t , attention weights α_{ti} are computed as: $\alpha_{ti} = \exp(\text{score}(s_t, h_i)) / \sum_j \exp(\text{score}(s_t, h_j))$, where $\text{score}()$ is a learned compatibility function. The context vector is then $c_t = \sum_i \alpha_{ti} h_i$, providing targeted information for generating the next target word.

The Transformer architecture took this concept further by building models based entirely on attention mechanisms, eliminating recurrence altogether. Multi-head self-attention in both encoder and decoder, combined with cross-attention linking the two, enables the model to capture complex relationships between source and target sequences. This architecture has set new benchmarks in translation quality whilst dramatically reducing training time through parallelisation. The attention weights themselves provide interpretable alignment information, showing which source words influenced each target word—a feature that aids in debugging and understanding model behaviour. Modern translation systems incorporating these innovations approach or exceed human parity for well-resourced language pairs, though challenges remain for low-resource languages, domain-specific translation, and preserving subtle semantic nuances.

Attention over Images and Hierarchical Attention Models



Attention mechanisms, having proven their worth in sequential domains, extend naturally to spatial visual data through attention over images. This paradigm enables models to selectively focus on specific image regions, mimicking the human visual attention system that rapidly directs focus to salient areas. Rather than processing all image features uniformly, spatial attention computes importance weights for different regions, allowing the model to emphasise relevant areas whilst down-weighting irrelevant background information.

In image attention, a CNN extracts a spatial feature map where each position corresponds to a receptive field in the original image. The attention mechanism then computes a weight for each spatial location, typically by learning a compatibility function between the current decoder state (in sequence generation tasks) or query vector (in retrieval tasks) and each feature location. These weights form an attention map that can be visualised as a heatmap overlay on the original image, providing intuitive insights into which regions the model considers important for the current prediction.



Spatial Attention

Bottom-up attention focuses on salient image regions, computed from convolutional feature maps with spatial resolution preserved



Object-Level Attention

Mid-level attention operates on detected objects or regions of interest, attending to semantically meaningful units



Attribute Attention

Top-down attention focuses on specific object attributes or relationships, enabling fine-grained reasoning

Hierarchical attention models combine multiple attention layers operating at different levels of abstraction, creating a compositional attention structure that mirrors the hierarchical nature of visual understanding. A hierarchical model might first attend to objects or regions within an image, then attend to attributes or parts within those objects, and finally attend to relationships between objects. This multi-level approach improves both performance and interpretability in complex visual reasoning tasks such as visual question answering, where understanding requires integrating information across multiple spatial scales and semantic levels.

For instance, in image captioning, a two-level hierarchical attention might first identify relevant objects in the scene using object-level attention, then apply spatial attention within each object to capture fine-grained details. When generating the caption "a grey tabby cat sitting on a wooden bench", object-level attention highlights the cat and bench regions, whilst spatial attention within the cat region focuses on colour and pattern when generating "grey tabby", and on posture when generating "sitting". This hierarchical structure enables more nuanced and accurate descriptions compared to flat attention mechanisms.

Practical implementation of hierarchical attention in frameworks like TensorFlow or PyTorch involves composing multiple attention modules, each operating on features at different levels of the visual hierarchy. Object detection networks like Faster R-CNN can provide object-level features, which then serve as input to region-based attention mechanisms. Attention visualisations—heatmaps showing model focus at each hierarchy level—prove invaluable for debugging and understanding model behaviour, revealing whether the model attends to semantically meaningful regions or exploits spurious correlations in the training data.

Monte Carlo Methods in Machine Learning

Monte Carlo methods constitute a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. Named after the famous casino in Monaco, these techniques are particularly valuable when dealing with complex integrals, high-dimensional distributions, or problems where analytical solutions are intractable. In machine learning, Monte Carlo methods play crucial roles in Bayesian inference, reinforcement learning, probabilistic model training, and uncertainty quantification.

The fundamental principle underlying Monte Carlo methods is the law of large numbers: as the number of random samples increases, their empirical average converges to the true expected value. For estimating an expectation $E[f(x)]$ where x is drawn from some distribution $p(x)$, Monte Carlo approximation draws N samples $x_1, \dots, x_n \sim p(x)$ and approximates the expectation as $(1/N) \sum_i f(x_i)$. This simple yet powerful idea extends to estimating complex integrals, sampling from posterior distributions in Bayesian models, and computing gradients in models with discrete latent variables where backpropagation is not directly applicable.

01	02	03
Define Target Distribution	Generate Random Samples	Evaluate Function
Specify the distribution $p(x)$ to sample from and the function $f(x)$ whose expectation we wish to estimate	Draw independent samples $x_1, \dots, x_n$ from the distribution $p(x)$ using appropriate sampling techniques	Compute $f(x_i)$ for each sampled point, accumulating the results for averaging
04	05	
Compute Estimate	Assess Convergence	
Average the function values to obtain Monte Carlo estimate: $(1/N) \sum_i f(x_i) \rightarrow E[f(x)]$	Evaluate estimation quality; standard error decreases as $O(1/\sqrt{N})$ with sample size	

A concrete application in machine learning is Monte Carlo Tree Search (MCTS), which has achieved remarkable success in game playing and sequential decision-making. In the context of natural language generation, MCTS can guide decoding by building a search tree through repeated simulation, balancing exploration of novel sequences with exploitation of high-scoring paths. For image caption generation, MCTS can improve caption quality by looking ahead multiple steps and selecting words that lead to globally coherent descriptions rather than greedily optimising at each step.

```
import numpy as np

def monte_carlo_integration(f, n_samples=10000):
    """
    Estimate integral of f(x) over [0,1] using Monte Carlo sampling.

    Args:
        f: Function to integrate
        n_samples: Number of random samples

    Returns:
        Estimated integral value and standard error
    """
    # Generate uniform random samples in [0,1]
    samples = np.random.uniform(0, 1, n_samples)

    # Evaluate function at sample points
    f_values = f(samples)

    # Monte Carlo estimate: average of f(x)
    estimate = np.mean(f_values)

    # Standard error: std / sqrt(N)
    std_error = np.std(f_values) / np.sqrt(n_samples)

    return estimate, std_error

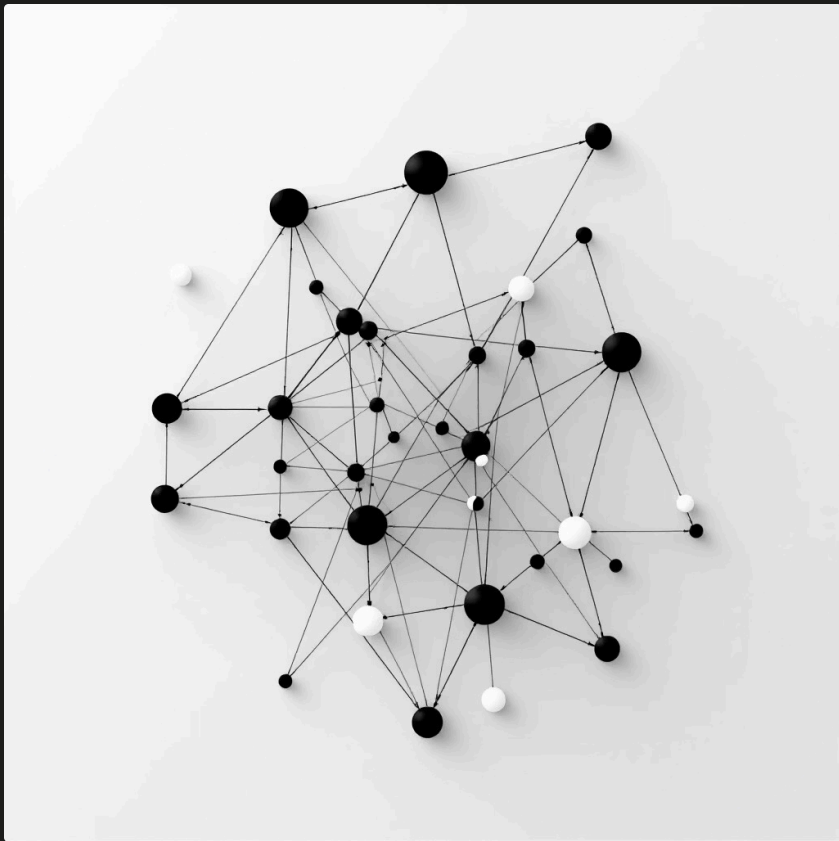
# Example: estimate integral of x^2 from 0 to 1
# True value: 1/3 ≈ 0.3333
result, error = monte_carlo_integration(lambda x: x**2, n_samples=100000)
print(f"Estimate: {result:.4f} ± {error:.4f}")
print(f"True value: {1/3:.4f}")
```

Monte Carlo methods face challenges in high-dimensional spaces, where the effective sample size required for accurate estimation grows exponentially with dimensionality—the so-called curse of dimensionality. Variance reduction techniques address this limitation through importance sampling, stratified sampling, and control variates, which reduce estimation variance without increasing sample size. In deep learning, reparameterisation tricks enable gradient estimation through stochastic nodes, whilst techniques like Markov Chain Monte Carlo (MCMC) enable sampling from complex posterior distributions that cannot be sampled directly. These sophisticated sampling strategies have become indispensable tools in modern probabilistic machine learning, enabling tractable inference in models that would otherwise be computationally infeasible.



Local Independencies in Markov Networks and Joint Distributions

Markov networks, also known as Markov Random Fields (MRFs) or undirected graphical models, provide a powerful framework for representing complex joint probability distributions through local conditional independence structures. Unlike directed graphical models (Bayesian networks), which use directed edges to represent causal relationships, Markov networks employ undirected edges to capture symmetric dependencies between variables. This undirected representation proves particularly natural for modelling spatial relationships, such as in image segmentation, or for capturing mutual dependencies that lack clear causal direction.



The fundamental property of Markov networks is the Markov blanket: a variable is conditionally independent of all other variables given its immediate neighbours in the graph. Formally, for a variable X_i with neighbours $N(i)$, we have $P(X_i | X_{-i}) = P(X_i | X_{N(i)})$, where X_{-i} denotes all variables except X_i . This local independence property enables tractable inference and learning despite potentially complex global dependencies.

The joint distribution represented by a Markov network factorises according to the cliques (fully connected subgraphs) in the network. Each clique c has an associated potential function $\psi_c(X_c)$ that assigns non-negative values to configurations of variables in that clique. The joint distribution is then proportional to the product of all clique potentials: $P(X) = (1/Z) \prod_c \psi_c(X_c)$, where Z is the partition function ensuring normalisation: $Z = \sum_x \prod_c \psi_c(x_c)$. Computing Z requires summing over all possible variable configurations, which is generally intractable for large networks, making approximate inference techniques essential.

1

Define Graph Structure

Construct undirected graph with variables as nodes and dependencies as edges, identifying all maximal cliques

2

Specify Clique Potentials

Assign non-negative potential functions $\psi_c(X_c)$ to each clique, encoding local interactions between variables

3

Factorize Joint Distribution

Express joint as $P(X) = (1/Z) \prod_c \psi_c(X_c)$, where product is over all cliques in the network

4

Compute Partition Function

Calculate normalisation constant $Z = \sum_x \prod_c \psi_c(x_c)$, summing over all configurations

5

Perform Inference

Use message passing, sampling, or variational methods to compute marginals and conditionals

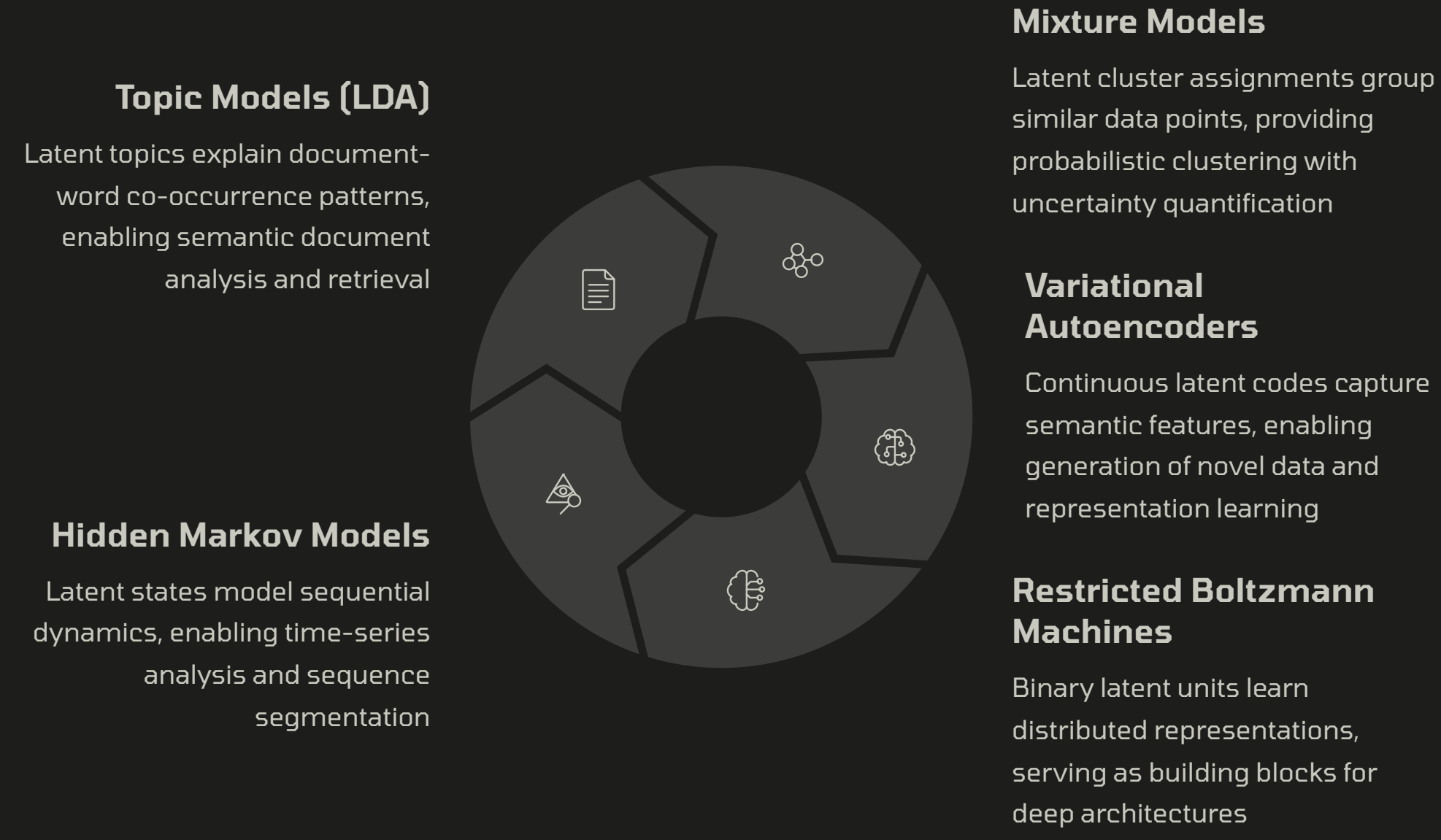
A canonical application of Markov networks is image segmentation, where the goal is to assign a label (e.g., object class) to each pixel. Variables X_{ij} represent pixel labels at position (i,j) , with edges connecting neighbouring pixels. Unary potentials $\psi_{ij}(X_{ij})$ encode the likelihood of each label based on local image features, whilst pairwise potentials $\psi_{ij,kl}(X_{ij}, X_{kl})$ encourage label smoothness, penalising label disagreements between adjacent pixels. This formulation captures the intuition that nearby pixels tend to have similar labels, whilst respecting evidence from image features. The joint distribution over all pixel labels factorises as $P(X) \propto \prod_{ij} \psi_{ij}(X_{ij}) \prod_{(ij) \sim (kl)} \psi_{ij,kl}(X_{ij}, X_{kl})$, where the second product is over neighbouring pixel pairs.

The mathematical elegance of Markov networks lies in how local independencies, encoded by graph structure, induce a factorised representation of the joint distribution. This factorisation simplifies both inference—computing marginal or conditional distributions—and learning—estimating potential functions from data. Graph cuts, belief propagation, and variational mean field methods exploit this structure for efficient approximate inference. Understanding these local independence structures proves crucial for designing effective probabilistic models across domains from computer vision and spatial statistics to social network analysis and computational biology.

The Concept of Latent Variables in Probabilistic Models

Latent variables—unobserved variables inferred from observed data—constitute a cornerstone concept in probabilistic modelling and statistical machine learning. These hidden variables capture underlying structure, abstract representations, or causal factors that generate observed data but are not directly measured. The introduction of latent variables enables models to represent complex distributions more compactly, discover interpretable structure in data, and make predictions that generalise beyond surface-level patterns. From clustering algorithms that infer group memberships to deep generative models that learn hierarchical representations, latent variables provide a principled framework for unsupervised learning.

The power of latent variable models stems from their ability to explain observed data through simpler, lower-dimensional hidden structure. Consider a dataset of documents: whilst the observed data consists of word counts, latent variables might represent abstract topics like "politics", "sports", or "science". Each document is then generated by mixing these topics in different proportions. This latent topic structure provides a more compact and interpretable representation than the raw word counts, enabling tasks like document similarity, retrieval, and classification. Mathematically, latent variable models specify a joint distribution $P(X, Z)$ over observed variables X and latent variables Z , with the observed data likelihood obtained by marginalising over latent variables: $P(X) = \sum_z P(X, Z)$ or $\int P(X, Z) dz$ for discrete or continuous Z respectively.



Inference in latent variable models—computing the posterior distribution $P(Z|X)$ of latent variables given observed data—presents computational challenges. Exact inference via marginalisation is often intractable due to the exponential or infinite number of possible latent configurations. This necessitates approximate inference techniques: variational inference approximates the posterior with a simpler distribution by minimising KL divergence; expectation-maximisation (EM) alternates between computing expected sufficient statistics under the current posterior and maximising the likelihood; sampling methods like Gibbs sampling or Hamiltonian Monte Carlo draw samples from the posterior distribution. These inference algorithms enable practical learning and prediction in latent variable models despite their computational complexity.

Unsupervised Learning

Latent variables discover hidden structure without labelled data, enabling clustering, dimensionality reduction, and density estimation without supervision.

Representation Learning

Latent representations capture semantic features useful for downstream tasks, providing compact and meaningful data encodings.

Generative Modelling

Sampling from latent-to-observed mappings generates new data, enabling creative applications like image synthesis and data augmentation.

Missing Data Imputation

Treating missing values as latent variables enables principled inference, maintaining uncertainty about unobserved quantities.

The conceptual elegance of latent variables extends beyond specific model families, providing a unifying perspective on diverse machine learning approaches. Deep neural networks can be viewed as hierarchical latent variable models, where hidden layer activations are latent representations of input data. This perspective illuminates the connection between classical statistical methods and modern deep learning, highlighting how both paradigms leverage unobserved structure to explain complex data. As machine learning continues advancing towards more interpretable and reliable systems, latent variable models—with their explicit probabilistic semantics and principled treatment of uncertainty—remain indispensable tools for understanding and modelling the world's complexity.

Restricted Boltzmann Machines: Theory and Practical Implementation

Restricted Boltzmann Machines (RBMs) represent a class of energy-based generative stochastic neural networks that learn probability distributions over observed data through the interplay of visible and hidden layers. Introduced by Paul Smolensky in 1986 and popularised by Geoffrey Hinton in the 2000s, RBMs served as crucial building blocks for the deep learning revolution, particularly in unsupervised pretraining of deep networks. An RBM consists of a layer of visible units v representing observed data and a layer of hidden units h capturing latent features, connected by a bipartite graph—visible units connect only to hidden units and vice versa, with no intra-layer connections. This restricted connectivity makes inference and learning more tractable than in general Boltzmann machines.

The joint distribution over visible and hidden units in an RBM is defined through an energy function $E(v,h) = -a^T v - b^T h - v^T W h$, where a and b are bias vectors for visible and hidden units respectively, and W is the weight matrix connecting them. The probability of a configuration is given by the Boltzmann distribution: $P(v,h) = (1/Z) \exp(-E(v,h))$, where Z is the partition function. The restricted bipartite structure ensures that hidden units are conditionally independent given visible units, and vice versa: $P(h_j|v) = \prod_i P(h_j|v_i)$ and $P(v_i|h) = \prod_j P(v_i|h_j)$. For binary units, these conditionals are sigmoid functions: $P(h_j=1|v) = \sigma(b_j + \sum_i W_{ij}v_i)$ and $P(v_i=1|h) = \sigma(a_i + \sum_j W_{ij}h_j)$, where σ is the logistic sigmoid.

Bipartite Architecture Visible and hidden layers connect only to each other, with no intra-layer connections, enabling efficient block Gibbs sampling		Energy-Based Model Joint distribution defined via energy function, with lower energy configurations having higher probability
Conditional Independence Restricted structure yields factorial conditionals $P(h v)$ and $P(v h)$, crucial for efficient inference		Contrastive Divergence Training approximates intractable log-likelihood gradient using short Markov chain samples

Training an RBM requires maximising the log-likelihood of observed data, which involves computing gradients of the log partition function—a sum over exponentially many configurations. Contrastive Divergence (CD), introduced by Hinton, provides a practical approximation. Rather than running a Markov chain to equilibrium to obtain samples from the model distribution, CD initialises the chain at training data and runs only k steps (often $k=1$) of Gibbs sampling. The gradient approximation is $\nabla W \log P(v) \approx E_{\text{data}}[vh^T] - E_{\text{model}}[vh^T]$, where E_{data} is computed from data and E_{model} from k -step reconstructions. Despite theoretical limitations, CD works remarkably well in practice, enabling efficient training of RBMs on large datasets.

```
import numpy as np

class RBM:
    """
    Restricted Boltzmann Machine for binary data.

    Parameters:
        n_visible: Number of visible units
        n_hidden: Number of hidden units
        learning_rate: Learning rate for gradient descent
        n_epochs: Number of training epochs
    """

    def __init__(self, n_visible, n_hidden, learning_rate=0.01, n_epochs=10):
        self.n_visible = n_visible
        self.n_hidden = n_hidden
        self.learning_rate = learning_rate
        self.n_epochs = n_epochs

        # Initialize weights and biases
        self.W = np.random.randn(n_visible, n_hidden) * 0.01
        self.a = np.zeros(n_visible) # visible bias
        self.b = np.zeros(n_hidden)  # hidden bias

    def sigmoid(self, x):
        """Sigmoid activation function."""
        return 1 / (1 + np.exp(-x))

    def sample_hidden(self, v):
        """Sample hidden units given visible units."""
        h_prob = self.sigmoid(np.dot(v, self.W) + self.b)
        h_sample = (np.random.rand(*h_prob.shape) < h_prob).astype(float)
        return h_prob, h_sample

    def sample_visible(self, h):
        """Sample visible units given hidden units."""
        v_prob = self.sigmoid(np.dot(h, self.W.T) + self.a)
        v_sample = (np.random.rand(*v_prob.shape) < v_prob).astype(float)
        return v_prob, v_sample

    def contrastive_divergence(self, v_data, k=1):
        """
        Perform k-step Contrastive Divergence.

        Args:
            v_data: Training data batch
            k: Number of Gibbs sampling steps
        """
        # Positive phase: compute statistics from data
        h_prob, h_sample = self.sample_hidden(v_data)
        positive_grad = np.dot(v_data.T, h_prob)

        # Negative phase: k steps of Gibbs sampling
        v_sample = v_data
        for _ in range(k):
            _, h_sample = self.sample_hidden(v_sample)
            _, v_sample = self.sample_visible(h_sample)

        h_prob_neg, _ = self.sample_hidden(v_sample)
        negative_grad = np.dot(v_sample.T, h_prob_neg)

        # Update parameters
        self.W += self.learning_rate * (positive_grad - negative_grad) / v_data.shape[0]
        self.a += self.learning_rate * np.mean(v_data - v_sample, axis=0)
        self.b += self.learning_rate * np.mean(h_prob - h_prob_neg, axis=0)

    def fit(self, X, batch_size=32):
        """
        Train the RBM on data X.

        Args:
            X: Training data, shape (n_samples, n_visible)
            batch_size: Mini-batch size
        """
        n_samples = X.shape[0]
        for epoch in range(self.n_epochs):
            # Shuffle data
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]

            # Mini-batch training
            for i in range(0, n_samples, batch_size):
                batch = X_shuffled[i:i+batch_size]
                self.contrastive_divergence(batch)

            # Compute reconstruction error
            _, h = self.sample_hidden(X)
            v_recon, _ = self.sample_visible(h)
            error = np.mean((X - v_recon) ** 2)
            print(f"Epoch {epoch+1}/{self.n_epochs}, Reconstruction Error: {error:.4f}")

    def reconstruct(self, v):
        """Reconstruct visible units from input."""
        _, h = self.sample_hidden(v)
        v_recon, _ = self.sample_visible(h)
        return v_recon

# Example usage: Train RBM on binary MNIST-like data
if __name__ == "__main__":
    # Generate synthetic binary data (replace with real data)
    np.random.seed(42)
    n_samples, n_visible = 1000, 784 # 28x28 images
    X_train = (np.random.rand(n_samples, n_visible) > 0.5).astype(float)

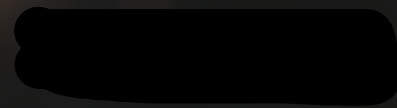
    # Create and train RBM
    rbm = RBM(n_visible=784, n_hidden=128, learning_rate=0.01, n_epochs=10)
    rbm.fit(X_train, batch_size=32)

    # Reconstruct test sample
    test_sample = X_train[:1]
    reconstruction = rbm.reconstruct(test_sample)
    print(f"Original: {test_sample[0][:10]}")
    print(f"Reconstructed: {reconstruction[0][:10]}")
```

RBMs find applications across diverse domains. In unsupervised feature learning, hidden units capture useful representations that can be fine-tuned for supervised tasks. In collaborative filtering, RBMs model user-item preferences, predicting ratings or recommendations. In generative modelling, RBMs serve as components in deep belief networks and deep Boltzmann machines, enabling layer-wise pretraining that initialises deep architectures favourably. Whilst modern alternatives like variational autoencoders and generative adversarial networks have superseded RBMs in many applications, understanding RBMs remains valuable both historically—as catalysts for the deep learning revolution—and conceptually, as exemplars of energy-based probabilistic modelling. Their elegant theoretical framework and practical training algorithms continue to inform developments in generative modelling and representation learning.

Understanding RBMs and GANs: Foundations, Architectures, and Applications

This comprehensive document explores the fascinating world of generative models, focusing on two groundbreaking approaches that have revolutionised machine learning: Restricted Boltzmann Machines (RBMs) and Generative Adversarial Networks (GANs). These powerful architectures have transformed how we approach unsupervised learning, data generation, and pattern recognition across countless domains.



Restricted Boltzmann Machines (RBMs) as Stochastic Neural Networks



Two-Layer Architecture

Visible layer for observed data and hidden layer for latent features, connected symmetrically



Probabilistic Activation

Neurons activate stochastically rather than deterministically



Unsupervised Learning

Powerful for feature extraction and dimensionality reduction tasks

Restricted Boltzmann Machines represent a landmark achievement in neural network architecture, originally introduced by Geoffrey Hinton and Terry Sejnowski during the 1980s. These sophisticated stochastic neural networks have fundamentally changed our understanding of how machines can learn patterns from data without explicit supervision. Unlike their deterministic counterparts, RBMs embrace uncertainty and probability as core principles, making them uniquely suited to modelling the complex, often ambiguous patterns found in real-world datasets.

The architecture of an RBM is elegantly simple yet remarkably powerful. It consists of two distinct layers: a visible layer that directly interfaces with observed data, and a hidden layer that captures abstract, latent features underlying that data. What makes RBMs "restricted" is the absence of connections within layers—neurons in the visible layer don't connect to each other, nor do neurons in the hidden layer. Instead, connections exist only between layers, creating a bipartite graph structure. This restriction, whilst seemingly limiting, actually enables efficient training algorithms and prevents the network from becoming overly complex.

The stochastic nature of RBMs is their defining characteristic and greatest strength. Rather than producing deterministic outputs, RBM neurons activate according to probability distributions. When presented with input data, each hidden unit calculates the probability of being active based on the weighted sum of visible unit activations. This probabilistic behaviour allows RBMs to model the inherent uncertainty in data and explore multiple possible representations rather than committing to a single interpretation.



Key Advantage

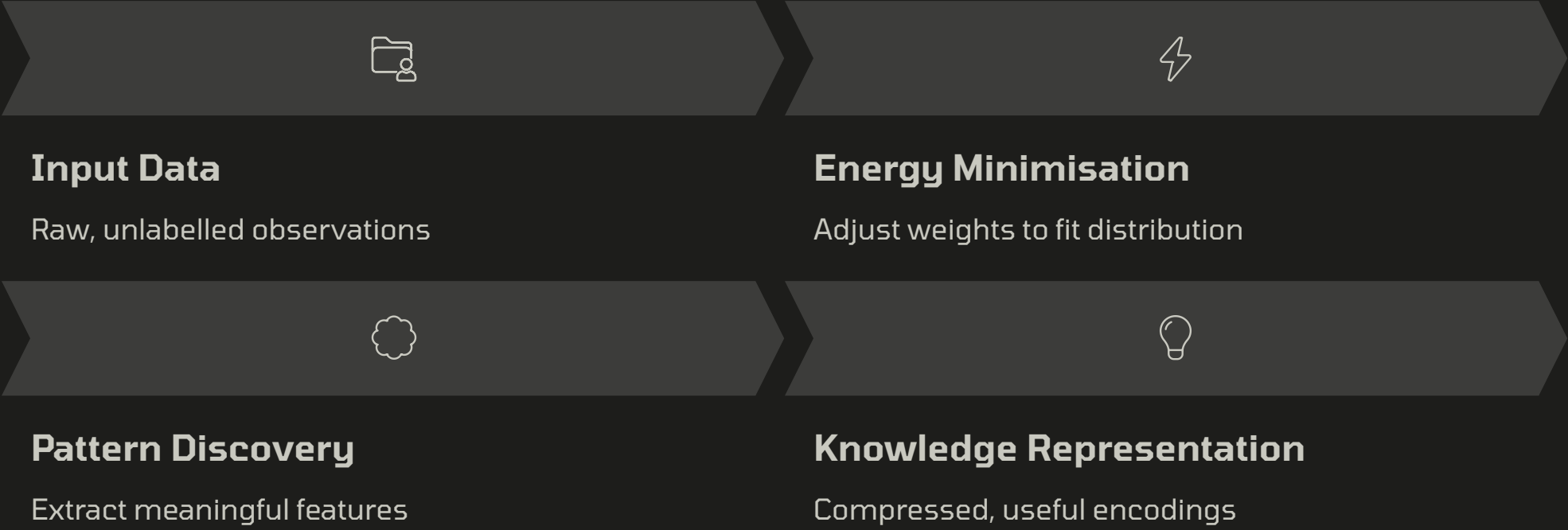
Stochasticity enables RBMs to escape local minima during training, leading to more robust learning outcomes.

This probabilistic framework makes RBMs exceptionally powerful for unsupervised learning tasks. They excel at feature extraction, automatically discovering meaningful patterns in unlabelled data. In dimensionality reduction applications, RBMs can compress high-dimensional data into compact representations whilst preserving essential structure. Perhaps most remarkably, because neurons activate stochastically, RBMs can explore different regions of the solution space during training, helping them avoid getting trapped in local minima—a common problem plaguing many optimisation algorithms.

The mathematical foundation of RBMs rests on energy-based models. The network assigns an energy value to each possible configuration of visible and hidden units, with lower energy corresponding to more probable configurations. Through training, the RBM adjusts its weights to lower the energy of observed data configurations whilst raising the energy of unlikely ones. This elegant formulation connects RBMs to statistical mechanics and provides a principled framework for learning probability distributions from data.

Unsupervised Learning with RBMs: Principles and Advantages

Unsupervised learning represents one of the most exciting frontiers in artificial intelligence, and Restricted Boltzmann Machines have emerged as particularly effective tools in this domain. Unlike supervised learning, which requires carefully labelled datasets—expensive and time-consuming to create—unsupervised learning allows machines to discover patterns and structure in raw, unlabelled data. This capability is transformative because unlabelled data is abundant, easily collected, and available in virtually every domain from web scraping to sensor readings.



The training process for RBMs centres on maximising the likelihood of observed data. In practical terms, this means adjusting the network's weights so that the configurations corresponding to actual data samples have low energy (high probability), whilst configurations that don't match real data have high energy (low probability). The network achieves this through an iterative process that gradually shapes its internal representation to capture the underlying structure of the training data. The mathematical objective is to minimise an energy function, a concept borrowed from physics that elegantly captures the notion of system stability.

Real-World Applications

- **Collaborative Filtering:** Netflix famously used RBMs in their recommendation system, where the network learned user preferences from viewing history to suggest relevant content
- **Deep Network Pretraining:** Before backpropagation improvements, RBMs were crucial for initialising deep neural networks, layer by layer
- **Anomaly Detection:** By learning normal data patterns, RBMs can identify unusual observations in cybersecurity and fraud detection
- **Feature Learning:** Automatically discovering useful representations for downstream tasks like classification or clustering



RBMs powered early recommendation systems by learning latent user preferences from viewing patterns

The advantages of RBMs over supervised methods extend beyond simply not requiring labels. Unsupervised pretraining with RBMs can dramatically improve the performance of subsequent supervised learning tasks by providing networks with initialised weights that already capture meaningful data structure. This approach was instrumental in the deep learning revolution, enabling training of networks with many layers before modern optimisation techniques became available. Additionally, because RBMs learn general data representations rather than task-specific mappings, the features they discover often transfer well across different problems within the same domain.

Implementation complexity represents another area where RBMs shine. Whilst the underlying mathematics involves probability theory and energy functions, the actual training algorithms—particularly Contrastive Divergence—are relatively straightforward to implement. This accessibility has made RBMs a popular choice for practitioners and researchers alike. Moreover, RBMs can be stacked into Deep Belief Networks, creating hierarchical representations where each layer captures increasingly abstract features, mirroring the hierarchical processing observed in biological vision systems.

Setting up a Markov Chain for RBMs: Sampling and Learning

The mathematical machinery underlying RBM training involves sophisticated sampling techniques rooted in probability theory and statistical mechanics. At the heart of this process lies the Markov Chain Monte Carlo (MCMC) method, a powerful approach for drawing samples from complex probability distributions that cannot be computed directly. Understanding this sampling process is crucial for grasping how RBMs actually learn from data and generate new samples.

01

Initialise Visible Units

Set visible layer to observed data sample

02

Sample Hidden Units

Compute hidden unit probabilities given visible units, then sample binary states

03

Reconstruct Visible Units

Sample visible units given the sampled hidden states

04

Iterate and Converge

Repeat sampling steps until chain reaches equilibrium distribution

Gibbs sampling, the specific MCMC variant used in RBMs, operates through a beautifully simple alternating process. Starting with an observed data sample on the visible layer, the algorithm computes the probability of each hidden unit being active based on the current visible states and the weights connecting them. These probabilities are then used to sample binary states for the hidden units—essentially "flipping coins" with biases determined by the computed probabilities. Next, the process reverses: using the sampled hidden states, the algorithm computes probabilities for visible units and samples their states accordingly.

This back-and-forth sampling creates a Markov chain—a sequence of states where each state depends only on the immediately preceding one. Remarkably, given enough iterations, this chain converges to the model's equilibrium distribution, meaning the samples it generates faithfully represent the probability distribution the RBM has learned. In theory, we should run this chain until convergence before each weight update during training. However, this would be computationally prohibitive, requiring potentially thousands of sampling steps per training example.

❏ Convergence Time

Full convergence might require hundreds or thousands of Gibbs sampling steps, making pure MCMC impractical for training.

Enter Contrastive Divergence (CD), an ingenious approximation algorithm that revolutionised RBM training. Proposed by Geoffrey Hinton, CD recognises that we don't actually need to run the Markov chain to full convergence. Instead, we can use very short chains—often just one complete cycle of sampling hidden then visible units (called CD-1)—to get sufficient gradient estimates for weight updates. This approximation works because we're starting the chain at actual data samples rather than random states, so even after just one step, the reconstructed samples contain useful information about how the model's current weights differ from the data distribution.

The brilliance of Contrastive Divergence lies in comparing the statistics of the original data with the statistics of these reconstructed samples. By measuring how the correlations between visible and hidden units change from the data to the reconstruction, CD estimates the gradient of the log-likelihood function—precisely what's needed to adjust weights in the right direction. This enables RBMs to train efficiently on large datasets whilst still capturing complex probability distributions. The stochastic sampling process also allows trained RBMs to generate new data samples, making them true generative models capable of creating novel examples that resemble their training data.

Introduction to Generative Adversarial Networks (GANs)

In 2014, Ian Goodfellow and colleagues introduced a radically different approach to generative modelling that would spark a revolution in artificial intelligence: Generative Adversarial Networks. This innovative architecture abandoned the energy-based frameworks and MCMC sampling of models like RBMs in favour of a game-theoretic approach inspired by economic theory and evolutionary biology. The core insight was elegantly simple yet profoundly powerful: instead of explicitly modelling probability distributions, why not pit two neural networks against each other in competition, letting them improve through adversarial training?

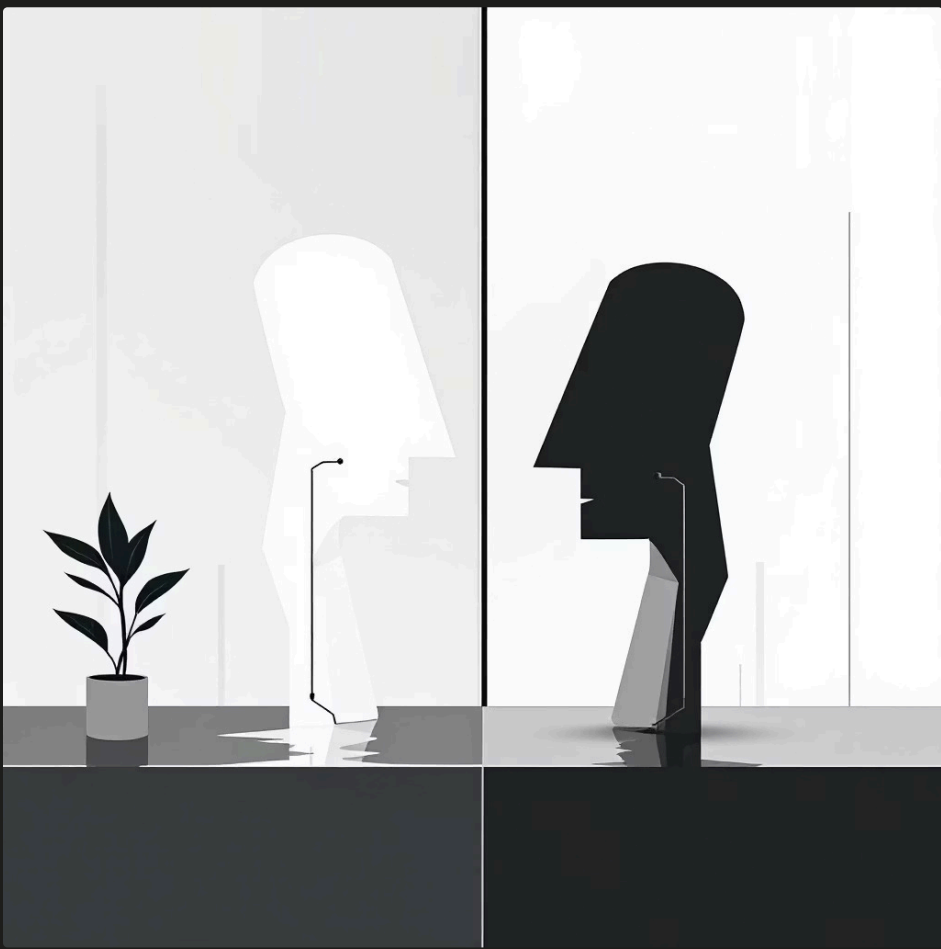
The Generator

Creates synthetic data from random noise vectors, attempting to produce samples indistinguishable from real data. Acts as a counterfeiter trying to create convincing fakes.

The Discriminator

Binary classifier distinguishing real data from generated samples. Acts as a detective trying to identify counterfeits.

The GAN framework consists of two neural networks locked in a perpetual contest. The Generator network functions as an artist or counterfeiter, learning to create synthetic data samples from random noise inputs. Starting with meaningless static, it gradually learns to produce increasingly realistic outputs through training. Meanwhile, the Discriminator network acts as a critic or detective, examining both real data samples and generated fakes, attempting to correctly classify which is which. The Discriminator outputs a probability—essentially its confidence that a given input is real rather than generated.



This setup creates what game theorists call a minimax game. The Generator aims to minimise the Discriminator's ability to detect fakes, whilst the Discriminator strives to maximise its classification accuracy. These opposing objectives drive both networks to improve iteratively. As the Discriminator becomes better at spotting fakes, it provides more informative feedback to the Generator. As the Generator produces more convincing samples, the Discriminator must become more sophisticated to maintain its edge.

The mathematical elegance of GANs lies in their objective function, which captures this adversarial dynamic. The Discriminator tries to maximise the probability of correctly classifying real and fake samples, whilst the Generator tries to minimise the Discriminator's success rate. At equilibrium—when neither network can improve further—the Generator produces samples indistinguishable from real data, and the Discriminator can do no better than random guessing. This theoretical endpoint represents a Nash equilibrium where each player's strategy is optimal given the other's strategy.

What makes GANs particularly exciting is their ability to generate remarkably realistic synthetic data without explicitly modelling probability distributions. Unlike RBMs, which learn energy functions and require careful sampling, GANs simply learn a direct mapping from noise to data space. This directness makes generation fast—just a single forward pass through the Generator—and produces high-quality outputs. The adversarial training framework has proven extraordinarily flexible, spawning countless variations and applications across domains from image synthesis to drug discovery. GANs represent a paradigm shift in how we think about generative modelling, replacing explicit probability estimation with implicit learning through competition.

GAN Architecture: Components and Training Dynamics

Understanding the architectural details and training mechanics of GANs reveals both their power and the challenges inherent in their design. Each component plays a crucial role in the delicate balance required for successful adversarial training, and the training dynamics themselves present unique optimisation challenges that distinguish GANs from traditional neural network training.

Generator Architecture

Maps latent space vectors to data space using deep convolutional layers, progressively upsampling from compact representations to full-resolution outputs

Discriminator Architecture

Binary classifier using convolutional layers for feature extraction, outputting probability that input is real versus generated

The Generator network begins with a latent space vector—typically sampled from a simple distribution like a standard normal or uniform distribution. This random noise vector, often 100-dimensional or more, serves as the seed from which synthetic data will be created. For image generation, the Generator typically employs a series of transposed convolutional layers (also called deconvolutions) that progressively upsample the input. Starting from a small spatial resolution, each layer doubles the height and width whilst reducing the number of channels, gradually transforming the compact latent vector into a full-resolution image. Batch normalisation and activation functions like ReLU are interspersed between layers to stabilise training and introduce non-linearity.



Latent Space Magic

The latent space captures semantic dimensions—moving through it smoothly interpolates between different generated outputs, revealing learned structure.

The Discriminator follows a more conventional deep learning architecture, essentially functioning as a convolutional neural network for binary classification. It receives either real data samples or generated fakes as input and processes them through a series of convolutional layers that progressively downsample the spatial dimensions whilst increasing feature channels. These layers extract hierarchical features, from simple edges in early layers to complex textures and structures in deeper layers. The final layer typically uses a sigmoid activation to output a probability between 0 and 1, representing the network's confidence that the input is real.

Training GANs involves a carefully choreographed alternating optimisation process. In each training iteration, we first update the Discriminator. We sample a batch of real data and a batch of random noise vectors. The noise is passed through the Generator to create fake samples. The Discriminator then evaluates both real and fake samples, computing its classification loss. Crucially, during this phase, we treat the Generator as frozen—its weights don't change—whilst we update the Discriminator's weights to maximise its classification accuracy. The Discriminator learns to output values near 1 for real samples and near 0 for fakes.

Next comes the Generator update. We sample fresh noise vectors and pass them through the Generator to create new fake samples. These fakes are then evaluated by the Discriminator—but now we freeze the Discriminator's weights and backpropagate through both networks to update only the Generator. The key insight is that we want the Discriminator to be fooled; we're training the Generator to maximise the Discriminator's error. In practice, instead of minimising $\log(1 - D(G(z)))$, we often maximise $\log(D(G(z)))$ for better gradient flow in early training when generated samples are obviously fake.

This alternating optimisation creates a delicate training dynamic. If the Discriminator becomes too powerful too quickly, it perfectly classifies all samples, providing gradients of zero to the Generator—effectively stopping its learning. Conversely, if the Generator overwhelms the Discriminator, the Discriminator's feedback becomes meaningless. Successful GAN training requires maintaining this balance, often through careful tuning of learning rates, update frequencies (e.g., updating the Discriminator multiple times per Generator update), and architectural choices. The objective function represents a minimax game where we seek the saddle point—simultaneously maximising with respect to the Discriminator and minimising with respect to the Generator—a challenging optimisation problem that has inspired numerous algorithmic innovations.

Popular GAN Variants and Architectural Innovations

Since their introduction, GANs have spawned an explosion of variants and architectural innovations, each addressing specific challenges or enabling new applications. These developments have expanded GANs from academic curiosities to practical tools deployed in production systems across industries. The diversity of GAN architectures reflects both the flexibility of the adversarial training framework and the creative ingenuity of the research community.

		
CycleGAN Breakthrough architecture enabling unpaired image-to-image translation. Uses cycle consistency loss to learn mappings between domains without paired training examples. Famous for style transfer applications like transforming horses into zebras, summer photos into winter scenes, or photographs into paintings in various artistic styles.	StyleGAN Developed by NVIDIA, StyleGAN generates photorealistic high-resolution images with unprecedented control over style attributes. Introduces style-based generator architecture with adaptive instance normalisation, enabling fine-grained control over features at different scales. Produces remarkably realistic human faces and has become the foundation for many commercial applications.	Conditional GAN Extends basic GANs by conditioning both Generator and Discriminator on additional information like class labels or data from other modalities. Enables controlled generation of class-specific samples, such as generating images of specific digits or objects on demand, making GANs more applicable to supervised learning scenarios.

CycleGAN in Detail

CycleGAN's innovation lies in its cycle consistency loss, which ensures that translating an image from domain A to domain B and back to A should recover the original image. This elegant constraint allows learning domain translations from unpaired data—you don't need matching image pairs, just two collections of images from different domains. This has democratised style transfer, making it accessible without expensive paired datasets.

Applications extend beyond artistic transformations to practical domains like medical imaging (translating between MRI modalities), satellite imagery enhancement, and even semantic photo editing. The key insight that cyclic transformations should be consistent has proven remarkably powerful across diverse problem domains.

StyleGAN deserves special attention for its architectural innovations and practical impact. Unlike conventional generators that inject noise at the input, StyleGAN introduces randomness at multiple scales throughout the network via learned affine transformations. This style-based approach separates high-level attributes (pose, identity) controlled by early layers from stochastic details (hair texture, skin pores) introduced by later layers. The result is unprecedented control over generation, enabling smooth interpolations between styles and fine-grained editing of specific attributes whilst preserving others. StyleGAN2 and StyleGAN3 further refined the architecture, addressing artefacts and improving alignment.

Conditional GANs (cGANs) represent another crucial innovation, extending the framework to supervised and semi-supervised settings. By conditioning both networks on additional information—class labels, text descriptions, or even images from related domains—cGANs enable controlled generation and cross-domain translation tasks. In image-to-image translation, for example, a cGAN can learn to convert semantic label maps into photorealistic images, enabling applications in autonomous driving simulation and architectural visualisation. Text-to-image GANs like AttnGAN use attention mechanisms to focus on relevant words when generating corresponding image regions, producing remarkably coherent images from textual descriptions. These architectural innovations have transformed GANs from proof-of-concept research into practical tools addressing real-world challenges.

Additional Notable Variants

- **DiscoGAN:** Similar to CycleGAN, discovers cross-domain relations without supervision
- **Progressive GAN:** Trains generator and discriminator progressively, starting at low resolution and gradually increasing detail
- **WGAN:** Uses Wasserstein distance instead of Jensen-Shannon divergence, providing more stable training
- **BigGAN:** Scales up GANs to unprecedented sizes, generating high-fidelity, diverse images across many classes
- **Text-to-Image GANs:** AttnGAN, StackGAN synthesise images from text descriptions

Applications of GANs: Real-World Examples and Use Cases

The practical applications of GANs have expanded far beyond their original image generation demonstrations, touching nearly every domain where synthetic data or learned transformations provide value. From creative industries to scientific research, GANs are solving real problems and enabling entirely new capabilities. Understanding these applications reveals both the maturity of GAN technology and the vast potential for future innovation.



Image Synthesis and Editing

GANs excel at generating photorealistic images from scratch. StyleGAN-generated faces are indistinguishable from photographs, whilst GANs can also create realistic landscapes, artworks, and designs. Beyond generation, GANs enable sophisticated editing: changing facial expressions, aging or de-aging portraits, or transferring artistic styles whilst preserving content. Applications include creating synthetic training data, generating product mockups, and assisting digital artists with AI-powered creative tools.



Data Augmentation

Medical imaging, autonomous driving, and other domains often face data scarcity challenges. GANs synthesise realistic additional training samples, expanding small datasets and improving classifier performance. In medical imaging, GANs generate synthetic CT scans or MRI images with rare pathologies, helping train diagnostic systems without compromising patient privacy. For autonomous vehicles, GANs create diverse driving scenarios—different weather, lighting, traffic conditions—improving model robustness without extensive real-world data collection.



Anomaly Detection

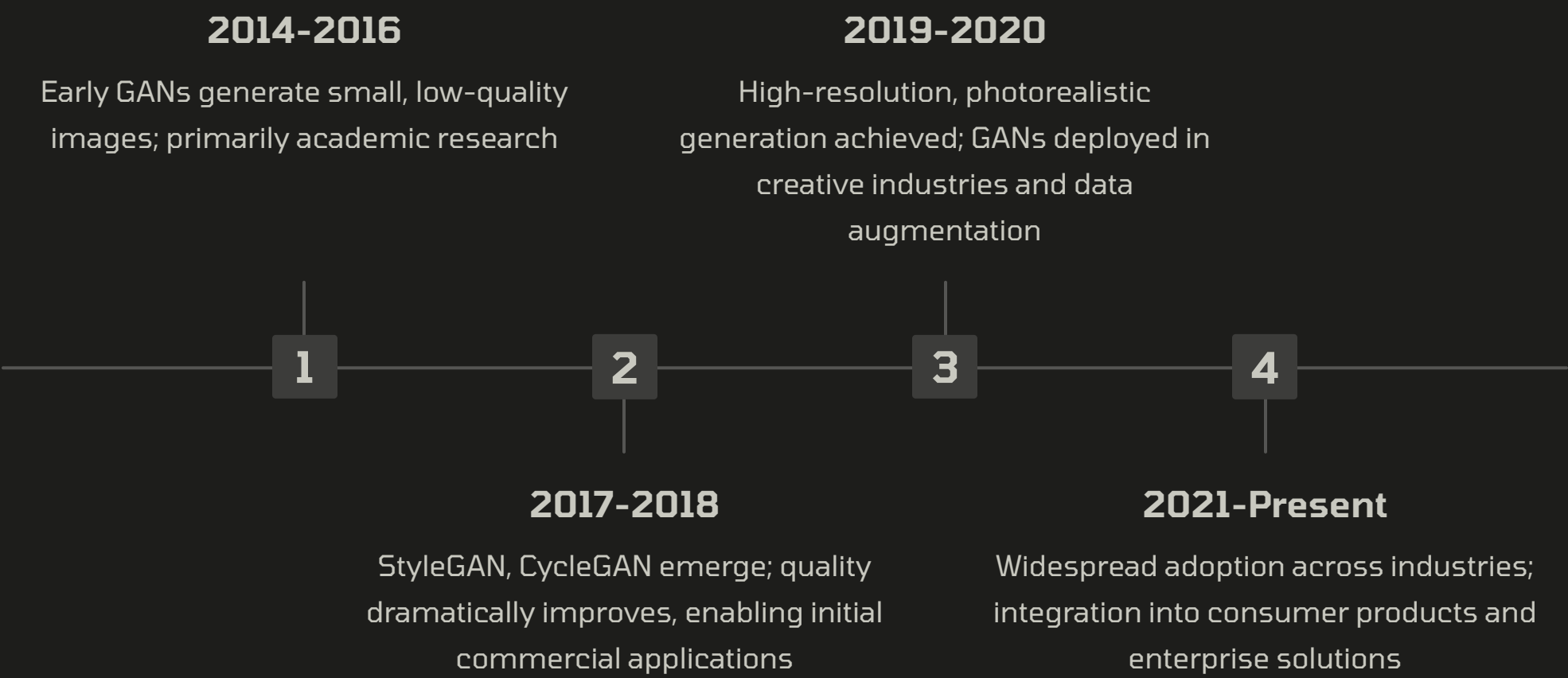
By learning distributions of normal data, GANs identify outliers and anomalies in cybersecurity, industrial monitoring, and fraud detection. The Discriminator distinguishes between typical patterns and unusual observations, whilst the Generator's reconstruction error reveals samples that don't fit the learned normal distribution. This approach detects network intrusions, identifies defective manufacturing products, and flags fraudulent transactions by recognising deviations from expected patterns.



Text-to-image GANs transform written descriptions into visual realisations

Text-to-Image Generation

Perhaps the most publicly visible GAN application, text-to-image generation has captured imagination worldwide. Models like DALL·E (technically not a pure GAN but inspired by adversarial training principles), AttnGAN, and others translate textual descriptions into corresponding images. This capability revolutionises creative workflows, enabling designers to rapidly prototype visual concepts from simple text prompts, generates illustrations for written content, and assists in conceptualising products or scenes that don't yet exist physically.



Beyond these major categories, GANs find applications in numerous specialised domains. In drug discovery, GANs generate novel molecular structures with desired properties, accelerating pharmaceutical research. In video games and film, GANs create textures, landscapes, and character variations, reducing manual content creation effort. Fashion design benefits from GANs that generate new clothing patterns and styles. Scientific visualisation employs GANs to enhance resolution of satellite imagery or microscopy data. Audio generation GANs synthesise realistic speech and music. The common thread across these applications is GANs' ability to learn complex data distributions and generate novel, realistic samples—a capability with virtually unlimited potential as the technology continues maturing.

The impact extends beyond technical capability to societal implications. Deepfakes—realistic face-swapped videos created using GANs—raise concerns about misinformation and authenticity. Meanwhile, synthetic data generation addresses privacy concerns by replacing sensitive real data with statistically similar but fabricated samples. As GANs become more sophisticated and accessible, navigating these opportunities and challenges will require ongoing dialogue between technologists, policymakers, and society at large. The applications explored here represent just the beginning of what generative adversarial networks will ultimately enable.

Example Programme: Simple GAN Implementation in Python (TensorFlow/Keras)

Understanding GAN theory comes alive through practical implementation. The following example demonstrates a minimal but complete GAN for generating simple images, providing a foundation you can extend and experiment with. This implementation uses TensorFlow/Keras, popular frameworks offering accessible APIs for building neural networks.

Implementation Overview

Our simple GAN will generate small synthetic images, learning to mimic patterns from training data. We'll build both Generator and Discriminator networks, define the adversarial training loop, and visualise results. This example prioritises clarity and educational value over state-of-the-art performance, making it ideal for understanding GAN mechanics.

Requirements

TensorFlow 2.x, NumPy, Matplotlib for visualisation. Compatible with Python 3.7+

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

# Define the Generator network
def build_generator(latent_dim):
    model = keras.Sequential([
        layers.Dense(7*7*128, input_dim=latent_dim),
        layers.Reshape((7, 7, 128)),
        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.2),

        layers.Conv2DTranspose(64, (5,5), strides=(2,2), padding='same'),
        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.2),

        layers.Conv2DTranspose(1, (5,5), strides=(2,2), padding='same', activation='tanh')
    ], name='generator')
    return model

# Define the Discriminator network
def build_discriminator(img_shape):
    model = keras.Sequential([
        layers.Conv2D(64, (5,5), strides=(2,2), padding='same', input_shape=img_shape),
        layers.LeakyReLU(alpha=0.2),
        layers.Dropout(0.3),

        layers.Conv2D(128, (5,5), strides=(2,2), padding='same'),
        layers.LeakyReLU(alpha=0.2),
        layers.Dropout(0.3),

        layers.Flatten(),
        layers.Dense(1, activation='sigmoid')
    ], name='discriminator')
    return model

# Hyperparameters
latent_dim = 100
img_shape = (28, 28, 1)
epochs = 10000
batch_size = 128

# Build models
generator = build_generator(latent_dim)
discriminator = build_discriminator(img_shape)

# Compile discriminator
discriminator.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Combined model (generator + discriminator)
discriminator.trainable = False
gan_input = layers.Input(shape=(latent_dim,))
fake_image = generator(gan_input)
gan_output = discriminator(fake_image)
gan = keras.Model(gan_input, gan_output, name='gan')

gan.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5),
    loss='binary_crossentropy'
)

# Load dataset (using MNIST as example)
(x_train, _) = keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 127.5 - 1.0 # Normalise to [-1, 1]
x_train = np.expand_dims(x_train, axis=-1)

# Training loop
def train_gan(epochs, batch_size):
    for epoch in range(epochs):
        # Train Discriminator
        idx = np.random.randint(0, x_train.shape[0], batch_size)
        real_images = x_train[idx]

        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        fake_images = generator.predict(noise)

        d_loss_real = discriminator.train_on_batch(real_images, np.ones((batch_size, 1)))
        d_loss_fake = discriminator.train_on_batch(fake_images, np.zeros((batch_size, 1)))
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train Generator
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        g_loss = gan.train_on_batch(noise, np.ones((batch_size, 1)))

        # Print progress
        if epoch % 1000 == 0:
            print(f"Epoch {epoch}, D Loss: {d_loss[0]:.4f}, G Loss: {g_loss:.4f}")
            generate_and_save_images(epoch)

# Function to visualise generated images
def generate_and_save_images(epoch, examples=16):
    noise = np.random.normal(0, 1, (examples, latent_dim))
    generated_images = generator.predict(noise)
    generated_images = 0.5 * generated_images + 0.5 # Rescale to [0, 1]

    fig, axes = plt.subplots(4, 4, figsize=(8, 8))
    for i, ax in enumerate(axes.flat):
        ax.imshow(generated_images[i, :, :, 0], cmap='gray')
        ax.axis('off')
    plt.savefig(f'gan_epoch_{epoch}.png')
    plt.close()

# Run training
train_gan(epochs, batch_size)
```

This implementation demonstrates core GAN concepts: separate Generator and Discriminator architectures, alternating training between them, and the adversarial objective. The Generator uses transposed convolutions to upsample from latent vectors to images, whilst the Discriminator employs standard convolutions for classification. Key training considerations include using LeakyReLU activations, batch normalisation for stability, and Adam optimiser with reduced learning rate and adjusted beta parameters—all common practices for successful GAN training.

Experimentation Ideas

- Modify architectures: add more layers, change filter sizes, experiment with different activation functions
- Adjust hyperparameters: learning rates, batch size, latent dimension
- Implement conditioning: extend to conditional GAN by feeding labels to both networks
- Try different datasets: generate faces (CelebA), objects (CIFAR-10), or custom images
- Add regularisation: spectral normalisation, gradient penalty for stability
- Visualise latent space: interpolate between points to see smooth transitions

Training Tips

GANs are notoriously finicky. If training fails, try reducing learning rates, increasing batch size, or adjusting update frequencies. Monitoring both losses helps diagnose issues.

This simple GAN provides a starting point for exploring generative modelling. Whilst it produces basic results, the same principles scale to sophisticated architectures like StyleGAN. Experimenting with this code—modifying architectures, adjusting hyperparameters, visualising intermediate outputs—builds intuition about adversarial training dynamics. The skills developed here transfer directly to implementing advanced variants and applying GANs to real-world problems. Generative models represent one of AI's most exciting frontiers, and hands-on implementation is the best way to truly understand their power and potential.